

VANCHI VIKASH P M

Software Engineer · Full-Stack | Applied ML | Security
Chennai, Tamil Nadu · +91 6382285582 · vikashvanchi@gmail.com
vanjivikash.vercel.app · linkedin.com/in/vanjiivikash · github.com/vanji-creator

SUMMARY

Engineer building applied ML and retrieval systems end to end — evaluation harness first, then measured improvement. Shipped an on-device URL classifier to the Chrome Web Store (98.6% accuracy, 0.3% FPR) and built a cross-lingual RAG system that lifted retrieval accuracy from 44 to 90 out of 100 across six independently measured changes, each validated for statistical significance. ~2 years production engineering on Government of India platforms serving 2,500+ users in Python/FastAPI, React and PostgreSQL.

SKILLS

ML & Retrieval: PyTorch, HuggingFace Transformers, sentence-transformers, scikit-learn, NumPy, LaBSE, cross-encoder reranking, BM25, HyDE, hybrid dense + sparse retrieval, RAG pipelines, embeddings, fine-tuning

Evaluation: Evaluation harness design, recall@k / precision@k / MRR, McNemar's exact test, ablation studies, data-leakage auditing, calibration analysis

LLMs: Sarvam, Google Gemini, Qwen3, Llama.cpp, prompt engineering, response caching, cost benchmarking

Languages: Python, TypeScript, JavaScript (ES6+), SQL

Backend & Data: FastAPI, Uvicorn, Node.js, REST API design, PostgreSQL, SQL, Firebase, WebSockets (Socket.IO)

Frontend: React.js, Next.js 15, Redux Toolkit, Tailwind CSS, TypeScript

Production Engineering: Docker, CI/CD, Git, Linux, structured JSONL logging, distributed request tracing, circuit breakers, graceful degradation, GCP, Vercel

Testing & Security: pytest, Jest, load/performance testing, VAPT, OWASP remediation, secret-handling audits

EXPERIENCE

Centre for Development of Advanced Computing (C-DAC), Chennai Jul 2024 – Present

Project Associate — Software Engineer

- Built the Python/FastAPI backend for the Supply Order Monitoring Portal, a national system serving 2,500+ users, including the analytics layer for delivery-delay prediction, demand forecasting and defaulter identification.
- Architected and shipped React/Next.js front-ends for two further production portals, and designed the data-exchange interface integrating the portal with an external inventory and logistics system — record mapping, API contract, error handling and reconciliation.
- Planned and executed test strategy across the stack: automated pytest and Jest unit tests, user-acceptance testing with client stakeholders, and load testing before rollout.
- Led security hardening — closed 25+ critical VAPT findings (XSS, CSRF, clickjacking) and implemented HSTS, CSP and SSL/TLS controls, earning CERT-In "Safe to Host" certification and cutting the assessed risk score by 85%.
- Owned CI/CD and production release engineering on on-premise Linux/Apache infrastructure, sustaining 99.9% uptime across the portal estate.

PROJECTS

Kural RAG — Cross-Lingual Retrieval over Classical Tamil | Python, sentence-transformers, LaBSE, FastAPI, Next.js 15, TypeScript

- Built a cross-lingual retrieval system that answers plain-English questions from 1,330 classical Tamil verses and cites the specific verses behind every claim. Improved retrieval accuracy from 44 to 90 out of 100 on a hand-labelled question set through six independently measured changes — a 105% relative gain.
- Designed and hand-labelled a 233-question evaluation harness before any optimisation, then validated every change with McNemar's exact test — and declined to claim a 15-point improvement that landed at $p = 0.0534$.
- Implemented hybrid retrieval combining LaBSE dense embeddings with a from-scratch BM25 index (per-query min-max normalised before fusion), cross-encoder reranking over the top 50, chapter-level signal blending, and HyDE query rewriting to close the vocabulary gap between modern questions and 19th-century translations.
- Cut per-query LLM cost 130× by benchmarking four providers against the project's own scorecard rather than defaulting to the best-known model; ~1.5 s end-to-end at ~US\$0.00005 per query.
- Found and corrected three data-leakage incidents in the evaluation pipeline, including 121 test questions leaked into a data file that had inflated a result to 97/100; ceiling analysis isolated the remaining headroom to first-stage recall rather than ranking.
- Built structured JSONL logging with distributed request tracing across the Next.js frontend and FastAPI service — per-stage latency breakdown, prompt-keyed persistent cache, and a circuit breaker that degrades to un-rewritten search with the degraded state surfaced to the user. Answer generation in progress.

Clikk — On-Device Malicious URL Detection | Python, scikit-learn, JavaScript, Chrome Extension

chromewebstore.google.com/detail/clikk · github.com/vanji-creator/linkguard

- Published on the Chrome Web Store; screens links for phishing and malware before they open, with the classifier running on-device so most URLs never leave the browser.
- Discarded a fine-tuned SecureBERT implementation after evaluation showed it had learned dataset shortcuts rather than URL structure and degraded on unseen domains; audited public training data against primary threat feeds, found incorrect labels in a widely used dataset, and rebuilt the training set from URLhaus, OpenPhish and ThreatFox.
- Shipped a TF-IDF character n-gram (3–5) + logistic regression pipeline chosen against real deployment constraints — offline, lightweight, in-browser: 98.6% accuracy at 0.3% FPR under domain-grouped evaluation. Reimplemented inference in dependency-free JavaScript and verified it against the Python pipeline across all 5,249 test URLs — identical predictions, max probability difference under $1e-7$, ~0.2 ms per URL.

EDUCATION

Amrita Vishwa Vidyapeetham — B.Tech, Computer Science and Engineering Oct 2020 – Apr 2024